

Anexo A. Derivaciones más a la izquierda

Gramática 1. Declaraciones del grafo

Ejemplo	Derivación más a la izquierda
grafo Lima dirigido { }	declGrafo ⇒ lm grafo id dirigidoOpt { elementos } ⇒ lm grafo id dirigido { elementos } ⇒ lm grafo id dirigido { }
nodo A (-12.046, -77.042);	declNodo ⇒ lm nodo id (coord , coord) ; ⇒ lm nodo id (- num , coord) ; ⇒ lm nodo id (- num , - num) ;
arista A -> B : 15 min tipo avenida;	declArista ⇒ lm arista id flecha id : peso unidad tipoOpt ; ⇒ lm arista id flecha id : num unidad tipoOpt ; ⇒ lm arista id flecha id : num unidad tipo id ;
grafo Cusco { nodo A (0, 0); nodo B (1, 1); arista A -> B : 3 km; }	declGrafo ⇒ lm grafo id dirigidoOpt { elementos } ⇒ lm grafo id { elementos } ⇒ lm grafo id { elemento elementos } ⇒ lm grafo id { declNodo elementos } ⇒ lm grafo id { nodo id (coord , coord) ; elementos } ⇒ lm grafo id { nodo id (num , num) ; elementos } ⇒ lm grafo id { nodo id (num , num) ; elemento elementos } ⇒ lm grafo id { nodo id (num , num) ; declNodo elementos } ⇒ lm grafo id { nodo id (num , num) ; nodo id (coord , coord) ; elementos } ⇒ lm grafo id { nodo id (num , num) ; nodo id (num , num) ; elementos } ⇒ lm grafo id { nodo id (num , num) ; nodo id (num , num) ; elemento elementos } ⇒ lm grafo id { nodo id (num , num) ; nodo id (num , num) ; declArista elementos } ⇒ lm grafo id { nodo id (num , num) ; nodo id (num , num) ; arista id flecha id : peso unidad tipoOpt ; elementos } ⇒ lm grafo id { nodo id (num , num) ; nodo id (num , num) ; arista id flecha id : num unidad tipoOpt ; elementos } ⇒ lm grafo id { nodo id (num , num) ; nodo id (num , num) ; arista id flecha id : num unidad ; elementos } ⇒ lm grafo id { nodo id (num , num) ; nodo id (num , num) ; arista id flecha id : num unidad ; }

Gramática 2. Heurísticas

Ejemplo	Derivación más a la izquierda
heuristica Cero(n, meta) { retornar 0; }	declHeuristica ⇒ lm heuristica id (id , id) { retornar expr ; } ⇒ lm heuristica id (id , id) { retornar factor ; } ⇒ lm heuristica id (id , id) { retornar num ; }
heuristica Manhattan(actual, meta) { retornar abs(actual.lat - meta.lat); }	declHeuristica ⇒ lm heuristica id (id , id) { retornar expr ; } ⇒ lm heuristica id (id , id) { retornar factor ; } ⇒ lm heuristica id (id , id) { retornar llamada ; } ⇒ lm heuristica id (id , id) { retornar id (argumentos) ; } ⇒ lm heuristica id (id , id) { retornar id (expr masArgumentos) ; } ⇒ lm heuristica id (id , id) { retornar id (suma masArgumentos) ; } ⇒ lm heuristica id (id , id) { retornar id (suma - termino masArgumentos) ; } ⇒ lm heuristica id (id , id) { retornar id (id . id - termino masArgumentos) ; } ⇒ lm heuristica id (id , id) { retornar id (id . id - id . id masArgumentos) ; } ⇒ lm heuristica id (id , id) { retornar id (id . id - id . id) ; }
heuristica Ponderada(g, h) { retornar g + h * 2; }	declHeuristica ⇒ lm heuristica id (id , id) { retornar expr ; } ⇒ lm heuristica id (id , id) { retornar suma ; } ⇒ lm heuristica id (id , id) { retornar suma + termino ; } ⇒ lm heuristica id (id , id) { retornar id + termino ; } ⇒ lm heuristica id (id , id) { retornar id + termino * factor ; } ⇒ lm heuristica id (id , id) { retornar id + id * factor ; } ⇒ lm heuristica id (id , id) { retornar id + id * num ; }
heuristica EsMeta(actual, meta) { retornar (actual == meta); }	declHeuristica ⇒ lm heuristica id (id , id) { retornar expr ; } ⇒ lm heuristica id (id , id) { retornar factor ; } ⇒ lm heuristica id (id , id) { retornar (expr) ; } ⇒ lm heuristica id (id , id) { retornar (expr oprel suma) ; } ⇒ lm heuristica id (id , id) { retornar (id oprel suma) ; } ⇒ lm heuristica id (id , id) { retornar (id oprel id) ; }

Gramática 3. Asignación y expresiones

Ejemplo	Derivación más a la izquierda
costo_g := 0;	asignacion ⇒ lm acceso := expr ; ⇒ lm id atributoOpt := expr ; ⇒ lm id := expr ; ⇒ lm id := factor ; ⇒ lm id := num ;
f_score := costo_g + estimacion_h * 2;	asignacion ⇒ lm acceso := expr ; ⇒ lm id atributoOpt := expr ; ⇒ lm id := expr ; ⇒ lm id := suma ; ⇒ lm id := suma + termino ;

	$\Rightarrow$ lm id := id + termino ; $\Rightarrow$ lm id := id + termino * factor ; $\Rightarrow$ lm id := id + id * factor ; $\Rightarrow$ lm id := id + id * num ;
es_meta := (nodo_actual == nodo_destino);	asignacion $\Rightarrow$ lm acceso := expr ; $\Rightarrow$ lm id atributoOpt := expr ; $\Rightarrow$ lm id := expr ; $\Rightarrow$ *lm id := factor ; $\Rightarrow$ lm id := (expr) ; $\Rightarrow$ lm id := (expr oprel suma) ; $\Rightarrow$ *lm id := (id oprel suma) ; $\Rightarrow$ *lm id := (id oprel id) ;
h := abs(actual.lat - meta.lat);	asignacion $\Rightarrow$ lm acceso := expr ; $\Rightarrow$ lm id atributoOpt := expr ; $\Rightarrow$ lm id := expr ; $\Rightarrow$ *lm id := factor ; $\Rightarrow$ lm id := llamada ; $\Rightarrow$ lm id := id (argumentos) ; $\Rightarrow$ lm id := id (expr masArgumentos) ; $\Rightarrow$ lm id := id (suma masArgumentos) ; $\Rightarrow$ lm id := id (suma - termino masArgumentos) ; $\Rightarrow$ *lm id := id (id . id - termino masArgumentos) ; $\Rightarrow$ *lm id := id (id . id - id . id masArgumentos) ; $\Rightarrow$ lm id := id (id . id - id . id) ;

Gramática 4. Reglas y sentencias selectivas

Ejemplo	Derivación más a la izquierda
regla Tarde { si hora entre 17:00 y 19:00 entonces { factor := 2; } sino { factor := 1; } }	declRegla $\Rightarrow$ lm regla id bloque $\Rightarrow$ lm regla id { instrucciones } $\Rightarrow$ lm regla id { instruccion instrucciones } $\Rightarrow$ lm regla id { siInstr instrucciones } $\Rightarrow$ lm regla id { si condicion entonces bloque sinoOpt instrucciones } $\Rightarrow$ lm regla id { si hora entre tiempo y tiempo entonces bloque sinoOpt instrucciones } $\Rightarrow$ lm regla id { si hora entre tiempo y tiempo entonces { instrucciones } sinoOpt instrucciones } $\Rightarrow$ lm regla id { si hora entre tiempo y tiempo entonces { instruccion instrucciones } sinoOpt instrucciones } $\Rightarrow$ lm regla id { si hora entre tiempo y tiempo entonces { asignacion instrucciones } sinoOpt instrucciones } $\Rightarrow$ *lm regla id { si hora entre tiempo y tiempo entonces { id := num ; instrucciones } sinoOpt instrucciones } $\Rightarrow$ lm regla id { si hora entre tiempo y tiempo entonces { id := num ; } sinoOpt instrucciones } $\Rightarrow$ lm regla id { si hora entre tiempo y tiempo entonces { id := num ; } sino bloque instrucciones } $\Rightarrow$ *lm regla id { si hora entre tiempo y tiempo entonces { id := num ; } sino { id := num ; } instrucciones } $\Rightarrow$ lm regla id { si hora entre tiempo y tiempo entonces { id := num ; } sino { id := num ; } }
regla Nocturna { si hora entre 23:00 y 05:00 entonces { factor := 3; } }	declRegla $\Rightarrow$ lm regla id bloque $\Rightarrow$ lm regla id { instrucciones } $\Rightarrow$ lm regla id { instruccion instrucciones } $\Rightarrow$ lm regla id { siInstr instrucciones } $\Rightarrow$ lm regla id { si condicion entonces bloque sinoOpt instrucciones } $\Rightarrow$ lm regla id { si hora entre tiempo y tiempo entonces bloque sinoOpt instrucciones } $\Rightarrow$ lm regla id { si hora entre tiempo y tiempo entonces { instrucciones } sinoOpt instrucciones } $\Rightarrow$ lm regla id { si hora entre tiempo y tiempo entonces { instruccion instrucciones } sinoOpt instrucciones } $\Rightarrow$ lm regla id { si hora entre tiempo y tiempo entonces { asignacion instrucciones } sinoOpt instrucciones } $\Rightarrow$ *lm regla id { si hora entre tiempo y tiempo entonces { id := num ; instrucciones } sinoOpt instrucciones } $\Rightarrow$ lm regla id { si hora entre tiempo y tiempo entonces { id := num ; } sinoOpt instrucciones } $\Rightarrow$ lm regla id { si hora entre tiempo y tiempo entonces { id := num ; } instrucciones } $\Rightarrow$ lm regla id { si hora entre tiempo y tiempo entonces { id := num ; } }
regla Buscar { si (nodo_actual == destino) entonces { encontrado := 1; } }	declRegla $\Rightarrow$ lm regla id bloque $\Rightarrow$ lm regla id { instrucciones } $\Rightarrow$ lm regla id { instruccion instrucciones } $\Rightarrow$ lm regla id { siInstr instrucciones } $\Rightarrow$ lm regla id { si condicion entonces bloque sinoOpt instrucciones } $\Rightarrow$ lm regla id { si expr entonces bloque sinoOpt instrucciones } $\Rightarrow$ *lm regla id { si (id oprel id) entonces bloque sinoOpt instrucciones } $\Rightarrow$ lm regla id { si (id oprel id) entonces { instrucciones } sinoOpt instrucciones } $\Rightarrow$ lm regla id { si (id oprel id) entonces { instruccion instrucciones } sinoOpt instrucciones } $\Rightarrow$ lm regla id { si (id oprel id) entonces { asignacion instrucciones } sinoOpt instrucciones } $\Rightarrow$ *lm regla id { si (id oprel id) entonces { id := num ; instrucciones } sinoOpt instrucciones } $\Rightarrow$ lm regla id { si (id oprel id) entonces { id := num ; } sinoOpt instrucciones } $\Rightarrow$ lm regla id { si (id oprel id) entonces { id := num ; } instrucciones } $\Rightarrow$ lm regla id { si (id oprel id) entonces { id := num ; } }
regla Prioridad { si (es_meta == 1) entonces { si (costo_g < limite) entonces { f_score := costo_g; } } sino	declRegla $\Rightarrow$ lm regla id bloque $\Rightarrow$ lm regla id { instrucciones } $\Rightarrow$ lm regla id { instruccion instrucciones } $\Rightarrow$ lm regla id { siInstr instrucciones } $\Rightarrow$ lm regla id { si condicion entonces bloque sinoOpt instrucciones } $\Rightarrow$ lm regla id { si expr entonces bloque sinoOpt instrucciones } $\Rightarrow$ *lm regla id { si (id oprel num) entonces bloque sinoOpt instrucciones } $\Rightarrow$ lm regla id { si (id oprel num) entonces { instrucciones } sinoOpt instrucciones } $\Rightarrow$ lm regla id { si (id oprel num) entonces { instruccion instrucciones } sinoOpt instrucciones }

<pre>{ f_score := 0; } }</pre>	<pre>⇒lm regla id { si (id oprel num) entonces { siInstr instrucciones } sinoOpt instrucciones } ⇒lm regla id { si (id oprel num) entonces { si condicion entonces bloque sinoOpt instrucciones } sinoOpt instrucciones } ⇒lm regla id { si (id oprel num) entonces { si expr entonces bloque sinoOpt instrucciones } sinoOpt instrucciones } ⇒*lm regla id { si (id oprel num) entonces { si (id oprel id) entonces bloque sinoOpt instrucciones } sinoOpt instrucciones } ⇒lm regla id { si (id oprel num) entonces { si (id oprel id) entonces { instrucciones } sinoOpt instrucciones } sinoOpt instrucciones } ⇒lm regla id { si (id oprel num) entonces { si (id oprel id) entonces { instruccion instrucciones } sinoOpt instrucciones } sinoOpt instrucciones } ⇒lm regla id { si (id oprel num) entonces { si (id oprel id) entonces { asignacion instrucciones } sinoOpt instrucciones } sinoOpt instrucciones } ⇒*lm regla id { si (id oprel num) entonces { si (id oprel id) entonces { id := id ; instrucciones } sinoOpt instrucciones } sinoOpt instrucciones } ⇒lm regla id { si (id oprel num) entonces { si (id oprel id) entonces { id := id ; } sinoOpt instrucciones } sinoOpt instrucciones } ⇒lm regla id { si (id oprel num) entonces { si (id oprel id) entonces { id := id ; } instrucciones } sinoOpt instrucciones } ⇒lm regla id { si (id oprel num) entonces { si (id oprel id) entonces { id := id ; } } sinoOpt instrucciones } ⇒lm regla id { si (id oprel num) entonces { si (id oprel id) entonces { id := id ; } } sino bloque instrucciones } ⇒*lm regla id { si (id oprel num) entonces { si (id oprel id) entonces { id := id ; } } sino { id := num ; } instrucciones } ⇒lm regla id { si (id oprel num) entonces { si (id oprel id) entonces { id := id ; } } sino { id := num ; } }</pre>
--------------------------------	---

Gramática 5. Sentencias iterativas

Ejemplo	Derivación más a la izquierda
para cada arista e de Lima { e.peso := 0; }	paraInstr ⇒lm para cada tipoElemento id de id bloque ⇒lm para cada arista id de id bloque ⇒lm para cada arista id de id { instrucciones } ⇒lm para cada arista id de id { instruccion instrucciones } ⇒lm para cada arista id de id { asignacion instrucciones } ⇒*lm para cada arista id de id { id . id := num ; instrucciones } ⇒lm para cada arista id de id { id . id := num ; }
para cada nodo n de Lima { n.visitado := 0; }	paraInstr ⇒lm para cada tipoElemento id de id bloque ⇒lm para cada nodo id de id bloque ⇒lm para cada nodo id de id { instrucciones } ⇒lm para cada nodo id de id { instruccion instrucciones } ⇒lm para cada nodo id de id { asignacion instrucciones } ⇒*lm para cada nodo id de id { id . id := num ; instrucciones } ⇒lm para cada nodo id de id { id . id := num ; }
para cada arista e de Lima { si (e.tipo == avenida) entonces { e.peso := 20; } }	paraInstr ⇒lm para cada tipoElemento id de id bloque ⇒lm para cada arista id de id bloque ⇒lm para cada arista id de id { instrucciones } ⇒lm para cada arista id de id { instruccion instrucciones } ⇒lm para cada arista id de id { siInstr instrucciones } ⇒lm para cada arista id de id { si condicion entonces bloque sinoOpt instrucciones } ⇒lm para cada arista id de id { si expr entonces bloque sinoOpt instrucciones } ⇒*lm para cada arista id de id { si (id . tipo oprel id) entonces bloque sinoOpt instrucciones } ⇒lm para cada arista id de id { si (id . tipo oprel id) entonces { instrucciones } sinoOpt instrucciones } ⇒lm para cada arista id de id { si (id . tipo oprel id) entonces { instruccion instrucciones } sinoOpt instrucciones } ⇒lm para cada arista id de id { si (id . tipo oprel id) entonces { asignacion instrucciones } sinoOpt instrucciones } ⇒*lm para cada arista id de id { si (id . tipo oprel id) entonces { id . id := num ; instrucciones } sinoOpt instrucciones } ⇒lm para cada arista id de id { si (id . tipo oprel id) entonces { id . id := num ; } sinoOpt instrucciones } ⇒lm para cada arista id de id { si (id . tipo oprel id) entonces { id . id := num ; } instrucciones } ⇒lm para cada arista id de id { si (id . tipo oprel id) entonces { id . id := num ; } }
para cada nodo n de Lima { para cada arista e de Lima { e.peso := e.peso * 2; } }	paraInstr ⇒lm para cada tipoElemento id de id bloque ⇒lm para cada nodo id de id bloque ⇒lm para cada nodo id de id { instrucciones } ⇒lm para cada nodo id de id { instruccion instrucciones } ⇒lm para cada nodo id de id { paraInstr instrucciones } ⇒lm para cada nodo id de id { para cada tipoElemento id de id bloque instrucciones } ⇒lm para cada nodo id de id { para cada arista id de id bloque instrucciones } ⇒lm para cada nodo id de id { para cada arista id de id { instrucciones } instrucciones } ⇒lm para cada nodo id de id { para cada arista id de id { instruccion instrucciones } instrucciones } ⇒lm para cada nodo id de id { para cada arista id de id { asignacion instrucciones } instrucciones } ⇒*lm para cada nodo id de id { para cada arista id de id { id . id := id . id * num ; instrucciones } instrucciones } ⇒lm para cada nodo id de id { para cada arista id de id { id . id := id . id * num ; } instrucciones }

⇒lm para cada nodo id de id { para cada arista id de id { id . id := id . id * num ; } }

Gramática 6. Consultas de ruteo y salida

Ejemplo	Derivación más a la izquierda
buscar ruta r desde Inicio hasta Fin en Lima usando Manhattan a las 18:00;	busqueda ⇒lm buscar ruta id desde id hasta id en id usando id conOpt horaOpt ; ⇒lm buscar ruta id desde id hasta id en id usando id horaOpt ; ⇒lm buscar ruta id desde id hasta id en id usando id a las tiempo ;
buscar ruta r desde A hasta B en Lima usando Euclidean con trafico;	busqueda ⇒lm buscar ruta id desde id hasta id en id usando id conOpt horaOpt ; ⇒lm buscar ruta id desde id hasta id en id usando id con id horaOpt ; ⇒lm buscar ruta id desde id hasta id en id usando id con id ;
buscar ruta r desde A hasta B en Lima usando Cero;	busqueda ⇒lm buscar ruta id desde id hasta id en id usando id conOpt horaOpt ; ⇒lm buscar ruta id desde id hasta id en id usando id horaOpt ; ⇒lm buscar ruta id desde id hasta id en id usando id ;
mostrar r;	mostrar ⇒lm mostrar id ;

Anexo B. Pruebas del analizador

Pruebas léxicas

Entradas válidas

N.º	Entrada	Tokens generados	Qué verifica
1	nodo A (-12.043, -77.028);	NODO(nodo) ID(A) PAR_A() MENOS(-) NUM(12.043) COMA(,) MENOS(-) NUM(77.028) PAR_C()) PUNTOYCOMA(;))	El signo es un token aparte y los decimales forman un solo NUM.
2	arista A <-> C : 2.5 km tipo avenida;	ARISTA(arista) ID(A) FLECHA(<->) ID(C) DOSP(:) NUM(2.5) UNIDAD(km) TIPO(tipo) ID(avenida) PUNTOYCOMA(;))	<-> es una sola FLECHA, y km es UNIDAD.
3	si hora entre 17:00 y 19:00 entonces { factor := 2; }	SI(si) HORA_KW(hora) ENTRE(entre) HORA(17:00) Y(y) HORA(19:00) ENTONCES(entonces) LLAVE_A({) ID(factor) ASIGNA(:=) NUM(2) PUNTOYCOMA(;) LLAVE_C(}))	17:00 es un solo HORA, y := gana a :.
4	buscar ruta r desde A hasta B en Lima usando Manhattan a las 18:00;	BUSCAR(buscar) RUTA(ruta) ID(r) DESDE(desde) ID(A) HASTA(hasta) ID(B) EN(en) ID(Lima) USANDO(usando) ID(Manhattan) A_LAS(a las) HORA(18:00) PUNTOYCOMA(;))	a las es un solo token, y Manhattan es un ID.
5	nodos := 4; // reinicia el contador	ID(nodos) ASIGNA(:=) NUM(4) PUNTOYCOMA(;))	nodos es ID aunque nodo es reservada, y el comentario se descarta.
6	es_meta := (f_score <= limite) != 1;	ID(es_meta) ASIGNA(:=) PAR_A() ID(f_score) OPREL(<=) ID(limite) PAR_C()) OPREL(!=) NUM(1) PUNTOYCOMA(;))	Operadores relacionales de dos caracteres.

Entradas inválidas

N.º	Entrada	Tokens generados	Qué verifica
7	nodo A (0, 0) @;	[línea 1:14] Error léxico: token recognition error at: '@'	Símbolo fuera del alfabeto. El lexer lo descarta y continúa.
8	costo := 3 # 4;	[línea 1:11] Error léxico: token recognition error at: '#'	Otro símbolo no reconocido.
9	mostrar "ruta";	[línea 1:8] ... at: '"' y [línea 1:13] ... at: '"'	PathLang no tiene cadenas de texto. Se reportan dos errores.
10	e.peso := a ! b;	[línea 1:12] Error léxico: token recognition error at: '!'	! solo es válido dentro de !=.

Pruebas sintácticas

Entradas válidas. Todos terminan con Programa válido: léxico y sintáctico OK.

N.º	Entrada	Gramática
1	grafo Cusco { nodo A (0, 0); nodo B (1, 1); arista A -> B : 3 km; }	G1
2	heurística Manhattan(actual, meta) { retornar abs(actual.lat - meta.lat); }	G2 y G3
3	regla Tarde { si hora entre 17:00 y 19:00 entonces { factor := 2; } sino { factor := 1; } }	G4
4	regla Anidada { si (a == 1) entonces { si (b < c) entonces { x := 1; } } sino { x := 2; } }	G4 (sino unido al si externo)
5	regla Reinicio { para cada nodo n de Lima { para cada arista e de Lima { e.peso := e.peso * 2; } } }	G5
6	buscar ruta r desde A hasta B en Lima usando Euclidean con trafico a las 08:30; seguido de mostrar r;	G6
7	grafo Lima dirigido { nodo A (-12.043, -77.028); nodo B (-12.046, -77.030); arista A -> B : 15 min tipo avenida; } heurística Manhattan(actual, meta) { retornar abs(actual.lat - meta.lat); } regla HoraPunta { para cada arista e de Lima { si hora entre 17:00 y 19:00 entonces { e.peso := e.peso * 2; } } } buscar ruta r desde A hasta B en Lima usando Manhattan a las 18:00; mostrar r;	

Entradas inválidas

N.º	Entrada	Salida	Error que se induce
8	grafo Lima { nodo A (0, 0) }	[línea 1:27] Error sintáctico: missing ';' at '}'	Falta ; en la declaración del nodo.
9	grafo Lima { ^d nodo A (0, 0); ^d	[línea 3:0] Error sintáctico: no viable alternative at input '<EOF>'	Falta la llave de cierre.
10	grafo Lima { arista A -> B : 15; }	[línea 1:31] Error sintáctico: missing UNIDAD at ';'	El peso no tiene unidad.
11	heurística H(a, b) { 0; }	[línea 1:21] Error sintáctico: missing 'retornar' at '0'	Falta retornar en la heurística.
12	regla R { si (a == b) { x := 1; } }	[línea 1:22] Error sintáctico: missing 'entonces' at '{'	Falta entonces.
13	regla R { x := 3 + ; }	[línea 1:19] Error sintáctico: mismatched input ';' expecting {NUM, '(', ID}	Expresión incompleta.
14	regla R { sino { x := 1; } }	[línea 1:10] ... no viable alternative at input 'sino' y [línea 1:27] ... input '}'	sino sin si.
15	buscar ruta r desde A hasta B en Lima;	[línea 1:37] Error sintáctico: mismatched input ';' expecting 'usando'	Falta la heurística.
16	regla R { x = 5; }	[línea 1:12] Error léxico: token recognition error at: '=' y [línea 1:14] Error sintáctico: missing ':=' at '5'	Se usó = en lugar de :=. Produce un error léxico y uno sintáctico.
17	regla R { para cada ruta r de Lima { } }	[línea 1:20] Error sintáctico: mismatched input 'ruta' expecting {'nodo', 'arista'}	Tipo de elemento inválido.